Improving Bash Generation in Small Language Models with Grammar-Constrained Decoding

May 08, 2026 +10 Like Discuss (0)

By Joseph Lucas

Bash is one of the most flexible and powerful interfaces exposed to AI agents. In the right system, a model that emits `grep`, `curl`, `tar`, or a shell pipeline is producing an executable action that can read files, mutate a workspace, open network connections, and chain tools together. For the NVIDIA AI Red Team, this makes command generation a useful research target. If smaller language models can be guided into valid, policy-aware command structures, they become more reliable components for agentic workflows that can be deployed into a wider range of environments.

Constrained decoding is a technique that modifies the sampling process in autoregressive language model generation. At each generation step, the model produces logits as normal, but before a token is selected, a grammar is applied to change the distribution (often by effectively blocking certain tokens).

PICARD used this technique to improve SQL generation, for example. The AI Red Team applied the same concept to Bash to improve the ability of small models to successfully achieve command-line tasks.

This post describes an experimental pipeline for generating Bash command grammars and applying them during decoding. We ran 13 small language models against 299 tasks and improved the average pass rate from 62.5% to 75.2%. The strongest result was on Qwen3-0.6B, where the pass rate increased from 16.7% to 59.2%.

Why Bash

Agentic systems increasingly use language models to generate code and commands that are executed by tools, shells, notebooks, build systems, and CI jobs. The security challenge isn't only whether the model "understands" a task. It is whether it can generate a syntactically valid action, scoped to the intended environment, and constrained away from unsafe forms.

Bash is a compact example of that problem:

- Syntax errors are unforgiving, and risk scales with task complexity.
- A valid command can still be operationally dangerous, such as a network command without a timeout or a destructive command with an overbroad path.
- Shell composition multiplies the state space. Pipes, redirects, command substitution, heredocs, loops, and conditionals all change what the model must emit and how a grammar would be applied.
- Small models often know the root binary to call but fail on exact syntax, argument order, quoting, control operators, or termination.
- Bash's expressiveness and power might make it the only tool needed if the model can be suitably expressive

The core research question was: Can constrained decoding improve small-model Bash command reliability enough to make them useful for agentic workflows?

Generating grammars

Handwriting a grammar for every command is brittle. Bash commands have many flags, aliases, optional values, positional arguments, and syntax variations. Instead, grammargen turns structured command evidence into Lark grammars.

The intermediate representation captures the pieces needed for constrained decoding, like:

- Command names and aliases.
- Boolean short flags and long flags.
- Valued flags, such as `-A 3` or `--max-count=10.`
- Positional arguments such as paths, patterns, words, and integers.
- Bounded repetition to keep the decoding state finite.

For example, a generated grep grammar includes a command-level `start` rule, a bounded option repetition, combined short flags, long flag alternatives, typed values, and shared terminals:

```
start: "grep" (WS grep_opt){0,8} WS WORD (WS PATH){0,5}
grep_opt: "-" /[EFGHILPRTUVZabchilnoqrsvwxz]+/
        | "-e" WS WORD
        | "-f" WS PATH
        | "-m" WS /[0-9]+/
        | "--ignore-case"
        | "--recursive"
        | "--regexp" ("=" | WS) WORD
        | "--file" ("=" | WS) PATH
        | "--max-count" ("=" | WS) /[0-9]+/
WORD: /[^\s|><&;()]{1,200}/
```

This grammar isn't intended to prove that every accepted command is safe. It defines a decoding boundary that restricts the model to tokens compliant with the grammar. Policy can then be encoded as additional grammar restrictions or applied as a separate control. `grammargen` will generate grammars from `--help` documentation or JSON tool schemas.

Applying grammars during decoding

These grammars can then be applied to llama.cpp inference through llguidance. Our evaluation focused on comparing native model performance with a "constrained retry" mode that used grammar-constrained decoding, then checked the output with tree-sitter-bash before executing.

If tree-sitter threw an error, we passed the error back as context into native mode so that we could at least have native-level performance. In this way, we could uplift model performance while still only executing one command in the test environment.

For example, prompted with "Base64 encode the contents of /workspace/plain.txt using openssl" we expect the model to follow `openssl` with `base64`, but the highest logit token for SmolLM2-360M-Instruct is `2`, which would result in a syntactically invalid command. With an openssl grammar applied as shown below, we instead get `base` as the next token (and autoregressively to `openssl base64` and eventually successful task completion).

```
start: "openssl" WS ssl_command

ssl_command: ssl_enc
           | ssl_dgst
           | ssl_rand
           | ssl_genrsa
           | ssl_req
           | ssl_x509
           | ssl_s_client
           | ssl_version

ssl_enc: ("enc" | "base64" | "aes-256-cbc" | "des3") (WS enc_opt){0,8}
ssl_dgst: ("dgst" | "sha256" | "sha512" | "md5") (WS dgst_opt){0,8}
ssl_rand: "rand" (WS rand_opt){0,8}
ssl_genrsa: "genrsa" (WS genrsa_opt){0,8}
ssl_req: "req" (WS req_opt){0,8}
ssl_x509: "x509" (WS x509_opt){0,8}
ssl_s_client: "s_client" (WS s_client_opt){0,8}
ssl_version: "version" (WS "-a"){0,2}

enc_opt: "-e" | "-d" | "-a" | "-base64"
       | "-aes-256-cbc" | "-aes-128-cbc" | "-des3" | "-des-ede3-cbc"
       | "-in" WS PATH | "-out" WS PATH
       | "-k" WS WORD | "-pass" WS WORD
       | "-salt" | "-nosalt" | "-pbkdf2"

dgst_opt: "-" /[a-z0-9]+/
        | "-out" WS PATH
        | PATH

rand_opt: "-hex" | "-base64" | "-out" WS PATH
        | /[0-9]+/

genrsa_opt: "-out" WS PATH | /[0-9]+/

req_opt: "-new" | "-x509" | "-nodes" | "-newkey" WS WORD
       | "-key" WS PATH | "-out" WS PATH
       | "-subj" WS SQ_STRING | "-days" WS /[0-9]+/

x509_opt: "-in" WS PATH | "-out" WS PATH | "-text" | "-noout"
        | "-dates" | "-subject" | "-issuer" | "-serial"
        | "-fingerprint" | "-inform" WS /[A-Z]+/ | "-outform" WS /[A-Z]+/

s_client_opt: "-connect" WS HOST_PORT
            | "-servername" WS WORD
            | "-showcerts"
            | "-CAfile" WS PATH
            | "-verify_return_error"
            | "-brief"

HOST_PORT: /[a-zA-Z0-9.\-]+:[0-9]+/
WORD: /[^\s|><&;]{1,200}/
```

Similarly, as shown below, grammar constrained decoding can reduce the common small model failure mode of early termination. In this case, the composed grammar prevented a pipe operator from being followed by a newline, and instead the model used `x` as the first token in `xargs`. Also, notice how with the grammar `cat` is in the top 5 logits, a good sign since piping to `cat` is a common operation.

```
gguf: SmolLM2-360M-Instruct.Q4_K_M.gguf
task: xargs_01
prompt: "Read filenames from /workspace/files.txt and delete them using xarg
canonical: cat /workspace/files.txt | xargs rm
assistant prefix: "cat /workspace/files.txt | "
grammar commands: ["cat", "xargs", "pipe"]
legal next tokens after mask: 37

native top logits
rank    token piece                                      logit
1         198 "\n"                                     17.3023
2        1792 " x"                                     16.1400
3         907 " #"                                     12.4901
4          33 "1"                                      12.4090
5         693 "xt"                                     12.3238

grammar-masked top logits
rank    token piece                                      logit
1        1792 " x"                                     16.1400
2         197 "\t"                                      9.6412
3         104 "x"                                       8.5847
4        2644 " cat"                                    7.3603
5         265 " c"                                      5.2345
```

Measuring uplift

Each model was evaluated on the same 299 tasks:

- Tier 1: 57 I/O primitive tasks
- Tier 2: 65 filter and transform tasks
- Tier 3: 139 recon and action tasks
- Tier 4: 38 shell construct tasks

The results are reported as pass rates. Table 1 compares native decoding to constrained decoding with tree-sitter retry.

Model	Native passed (out of 299)	Rate	Constrained passed (out of 299)	Rate	Uplift
Qwen3-0.6B	50	16.7%	177	59.2%	+42.5 pts
SmolLM2-360M-Instruct	88	29.4%	171	57.2%	+27.8 pts
Qwen2.5-0.5B-Instruct	133	44.5%	205	68.6%	+24.1 pts
Qwen3.5-0.8B	158	52.8%	200	66.9%	+14.0 pts
gemma-3n-E2B-it	190	63.5%	227	75.9%	+12.4 pts
SmolLM3-3B	207	69.2%	236	78.9%	+9.7 pts
gemma-4-E2B-it	213	71.2%	241	80.6%	+9.4 pts
Nemotron-3-Nano-4B	242	80.9%	264	88.3%	+7.4 pts
Phi-4-mini-instruct	225	75.3%	243	81.3%	+6.0 pts
Qwen3-1.7B	214	71.6%	229	76.6%	+5.0 pts
Qwen3-4B	234	78.3%	247	82.6%	+4.3 pts
Qwen3.5-4B	252	84.3%	258	86.3%	+2.0 pts
Qwen2.5-3B-Instruct	223	74.6%	226	75.6%	+1.0 pts

Table 1. Model performance and uplift from constrained decoding

Across all 13 models, constrained retry improved the mean pass rate from 62.5% to 75.2%. Every model improved overall, but the gains were largest for the smallest and weakest baselines, as shown in Figure 1. The tier-level averages in Table 2 show where the grammar helped most:

Tier	Native average	Constrained retry average	Average uplift
Tier 1: I/O primitives	79.8%	89.7%	+10.0 pts
Tier 2: Filter/transform	55.1%	72.5%	+17.4 pts
Tier 3: Recon/action	56.9%	72.2%	+15.3 pts
Tier 4: Shell constructs	69.4%	69.0%	-0.4 pts

Table 2. Model performance and uplift from constrained decoding by task tier complexity

Tier 4 involved tasks like chaining, backgrounding, and loops that required combinations of command grammars. Ultimately, the constrained generation was either too restrictive or too permissive to be helpful.

Figure 1. Task success by model and task tier, with uplift from constrained decoding shown in green and regression shown in red

Figure 2 shows task and model uplift vs regressions. Across 3,887 paired model-task results, constrained retry preserved 2,248 native passes, fixed 676 native failures, regressed 181 native passes, and left 782 failures unresolved.

In other words, the grammar path produced a net gain of 495 passing tasks across the full run, but did suffer some regressions based on the grammar conflicting with model bias when there were multiple ways to accomplish the task, or grammar incompleteness undermining the model's native capability.

Figure 2. Per-task success transition provided by constrained decoding

The grammar recovers many command syntax and surface-form failures in tiers 1-3. Tier 4 is harder with richer Bash constructs, such as multiline scripts, heredocs, loops, conditionals, command substitution, and process substitution, which need either richer grammars or a strategy that can selectively fall back to native generation.

What improved

The grammar helps most when the model already has the right intent but is likely to drift on syntax. It improves the selection of command names and flags, typed values, and end-of-turn handling.

Tree-sitter catch and retry adds a second layer. Even when constrained decoding produces malformed Bash because of a grammar gap or truncation, the evaluator can detect syntax errors before execution and ask for a corrected native output with the parse error included. This could be just one layer of error correction, depending on the system's constraints.

Security implications

Constrained decoding changes the probability distribution of the model's output before execution. That makes it useful, but only as one layer in an agentic AI control stack. The interesting security properties are:

- **Reliability as a security property**. Restricting the action surface can decrease uncertainty in the action space.
- **Policy encoded as syntax**. Grammars can forbid or require forms and arguments, such as excluding insecure flags or requiring timeouts.

The research also highlights a limitation. Generated grammars describe what a command accepts, but not what a specific model uses correctly. For broad commands such as curl, a grammar generated from help text may allow hundreds of legal flags. That is syntactically accurate, but too permissive to meaningfully improve reliability or enforce tradecraft.

This points toward learned or policy-refined grammars. Instead of accepting the entire legal command space, a learned grammar can encode the subset where a given model is reliable, plus hard safety rules such as HTTPS-only URLs, mandatory timeouts, or disallowed destructive flags.

Recommendations

For teams experimenting with grammar-constrained generation:

1. Start with a narrow benchmark. Measure native and constrained outputs on the same prompts before changing the grammar.
2. Validate grammars structurally and behaviorally. A grammar should parse, accept known-good commands, and reject known-bad examples.
3. Track regressions, not only uplift. While we showed a net increase in performance, our results show that constrained decoding can fight the model when the grammar can't express the intended structure.
4. Separate syntax success from task success. A syntactically valid command can still be semantically wrong or operationally unsafe.

Get started

Grammar-constrained decoding is a promising control for Bash-generating agents, especially when paired with execution-grounded evaluation and syntax validation. In our experiment, constrained retry improved the mean pass rate across 13 models from 62.5% to 75.2%, with the largest single-model gain on Qwen3-0.6B achieving a final task success close to models twice its size. The results also show that grammar constraints still struggle with richer shell constructs and composition.

To apply these ideas in your own agentic systems, treat grammar-constrained decoding as one control in a broader NVIDIA AI stack. Identify a small model, like NVIDIA Nemotron 3 Nano that performs well on your task and uplift it with constrained decoding.

To harden the system, evaluate programmable prompt, response, and agentic-security checks with NVIDIA NeMo Guardrails. The practical pattern is defense in depth: constrain the action grammar, sandbox execution with isolated hosts like Brev, measure native-to-constrained transitions, and promote only controls that improve reliability without hiding residual execution risk.

For more AI security research and guidance, follow the NVIDIA AI Red Team.

Discuss (0) +10 Like

Tags

Agentic AI / Generative AI | Trustworthy AI / Cybersecurity | General | Intermediate Technical | Deep Dive | AI Red Team | Featured | Security For AI

About the Authors

About Joseph Lucas

Joe is a Principal Offensive Security Researcher focused on AI at NVIDIA. He is the founder and Chair of the NumFOCUS Security Committee and is a member of the Jupyter Security Council. He was one of the architects and hosts of the DEF CON 30 AI Village Capture the Flag competition and is passionate about machine learning security education. He served in the US Army at US Cyber Command and the 101st Airborne Division. He holds a master's degree in Computer Science from Georgia Institute of Technology and a bachelor's degree in Mathematics from the United States Military Academy.

View all posts by Joseph Lucas >

Comments

Start the discussion at forums.developer.nvidia.com

Related posts

How to Train an AI Agent for Command-Line Tasks with Synthetic Data and Reinforcement Learning

Improve AI Code Generation Using NVIDIA NeMo Agent Toolkit

Rapidly Triage Container Security with the Vulnerability Analysis NVIDIA NIM Agent Blueprint

Secure LLM Tokenizers to Maintain Application Integrity

NVIDIA Enables Trustworthy, Safe, and Secure Large Language Model Conversational Systems

Watch NVIDIA CEO Jensen Huang's Keynote
June 1, 11 a.m. Taipei Time | Livestream
Add to Calendar
NVIDIA GTC

Copyright © 2026 NVIDIA Corporation